The RAG Gauntlet

A Developer's Journey from Fuzzy Search to Precise Answers

The Quest: A Smart Railway Assistant

We began with a single, massive text file containing over 13,000 distinct train schedules.

The goal was to build a system that could answer natural language questions about this data, effectively creating an intelligent railway enquiry assistant.

Our First Attempt: The ‘Vanilla’ RAG

Key Insight

The first critical decision was to treat each train schedule as a single, semantic unit. Instead of arbitrary character splits, we ensured **`1 Chunk = 1 Complete Train Schedule`.**

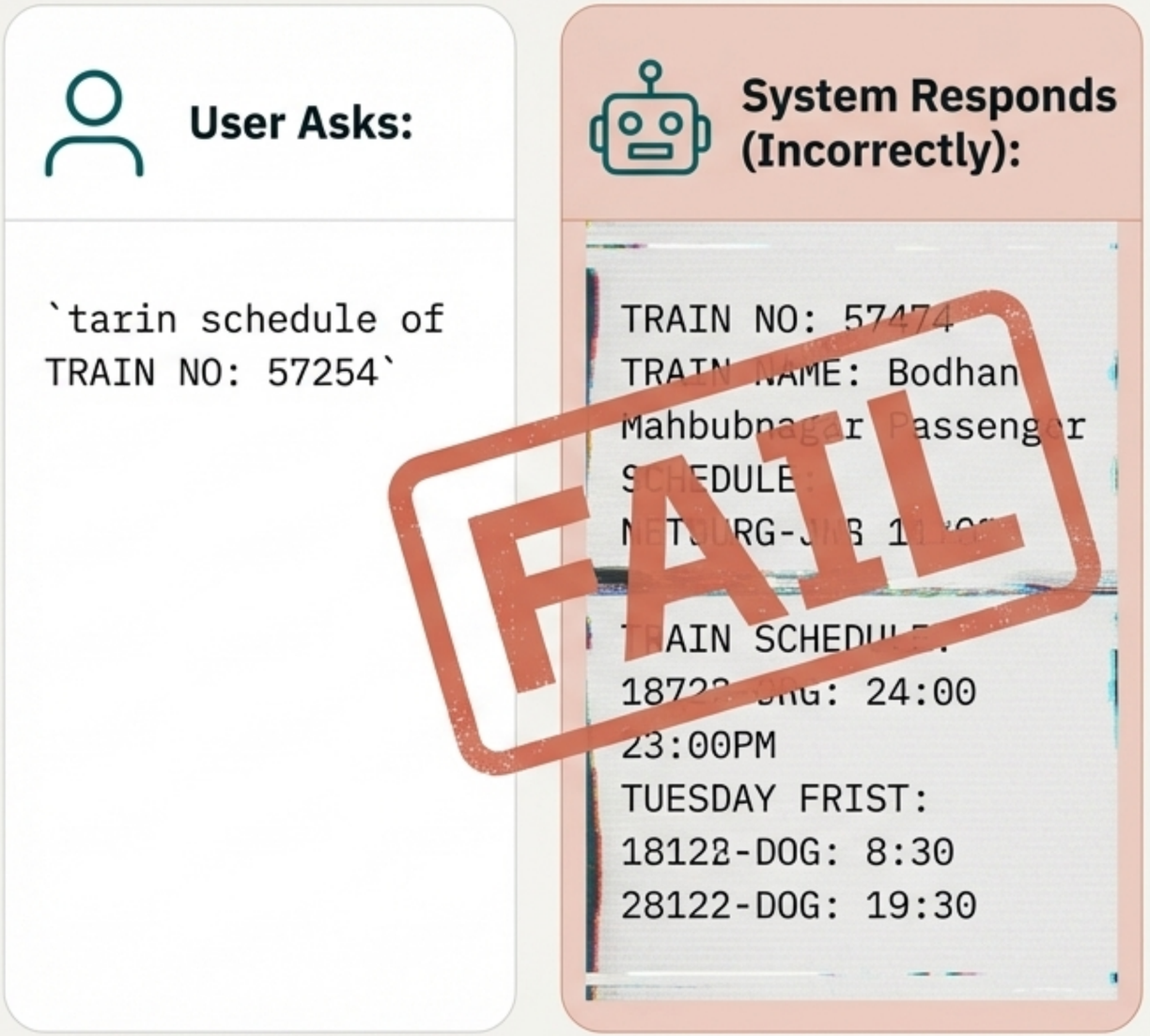
```
# 1. Read & Custom Chunk
full_text = read_text_file("All_Trains_Cleaned.txt")
train_chunks = chunk_by_train_schedule(full_text)

# 2. Embed & Store
vector_db = create_vector_store(train_chunks)

# 3. Retrieve & Generate
top_matches = retrieve_relevant_trains(vector_db, user_query)
answer = generate_humanized_answer(user_query, top_matches)
```

The First Trial: The Exact-Match Hydra Hydra

Our simple RAG immediately failed on a crucial task: looking up a specific train number.

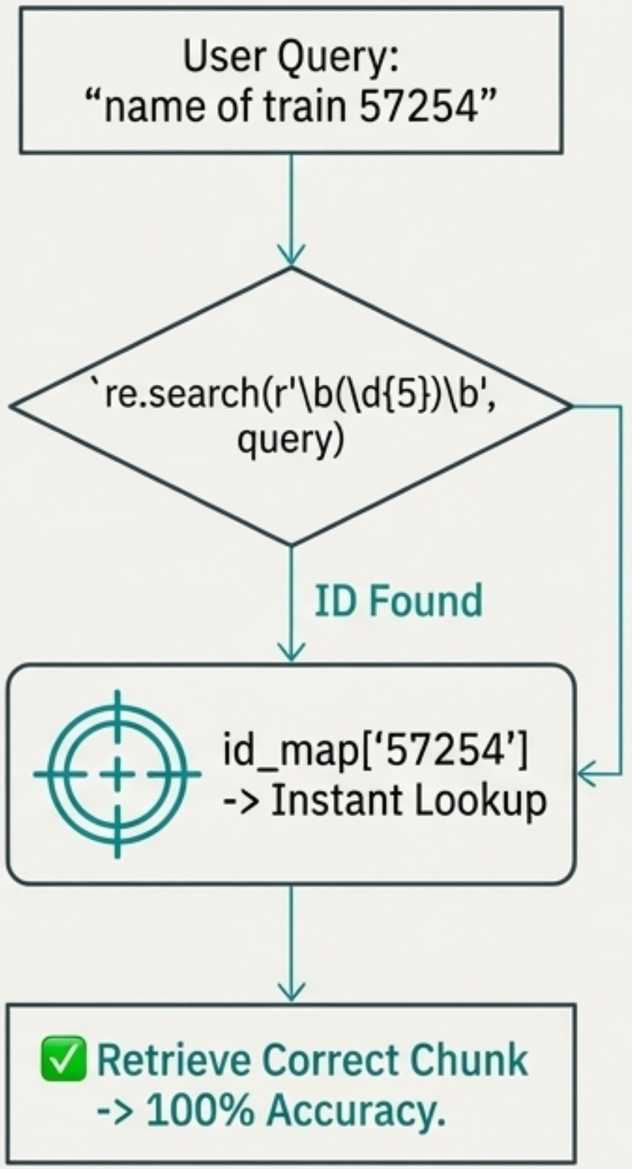
Vector search, designed for semantic meaning, struggled with the precision of IDs.

To a vector model, `57254` and `57474` look almost identical.



The Weapon: A Sniper Rifle for IDs

Instead of relying on AI for a database task, we implemented a “Hybrid Search.” We built a simple Python dictionary to map every train number to its specific chunk. If a query contains a 5-digit number, we bypass vector search entirely.

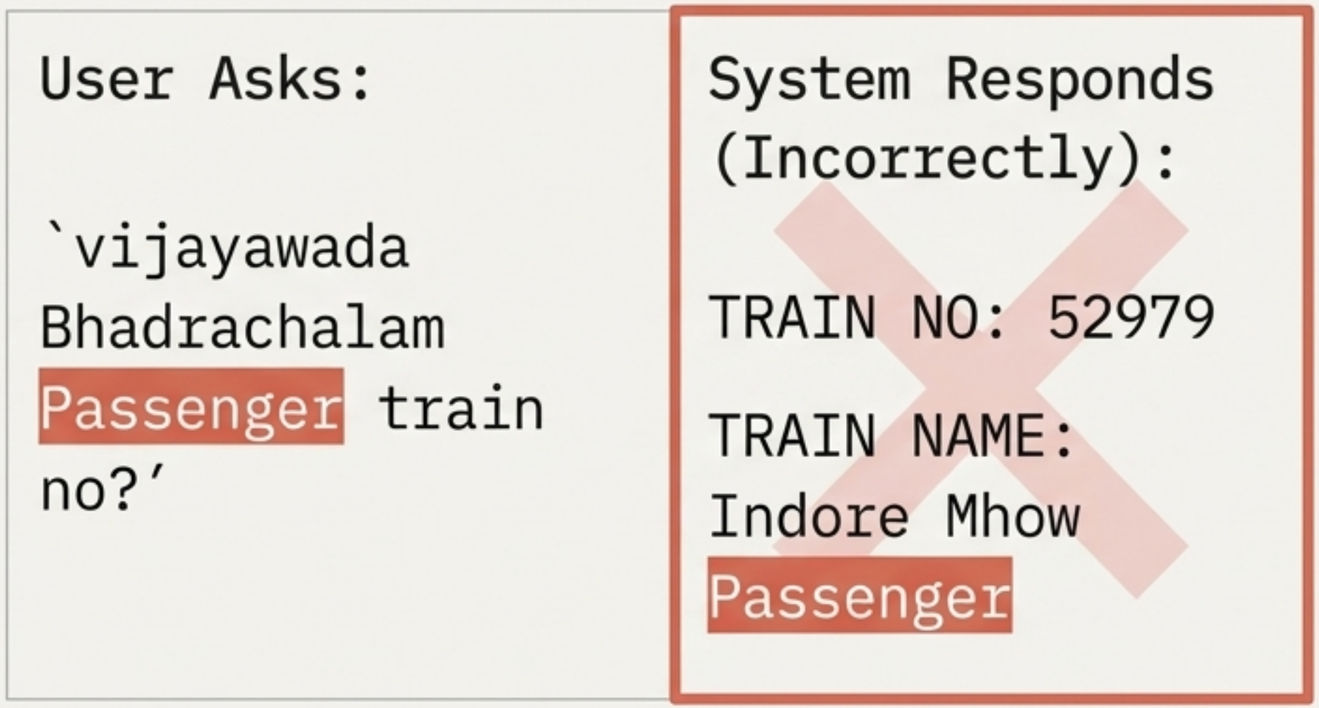



```
# Build an index for instant lookup
id_map = { "57254": chunk_304, ... }

def hybrid_search(query):
    # STRATEGY 1: EXACT ID MATCH
    id_match = re.search(r"\b(\d{5}\b", query)
    if id_match:
        train_id = id_match.group(1)
        if train_id in id_map:
            # Bypass vector search
            return [chunks[id_map[train_id]]]

    # STRATEGY 2: VECTOR SEARCH (Fallback)
    ...
```

The Second Trial: The Keyword Quagmire

The next failure

The next failure occurred when searching by name. The embedding model focused too much on the common word “Passenger” and failed to assign enough weight to the critical, unique keywords “Vijayawada” and “Bhadrachalam.”



The Weapon: A Magic Spell of Enrichment

We engineered the data before embedding. For each train, we created two versions: a **‘Display Chunk’** for the LLM to read, and a **‘Search Chunk’** for the vector model to index. The Search Chunk includes a descriptive header that forces the model to focus on the important keywords.

Display Chunk (For Reading)

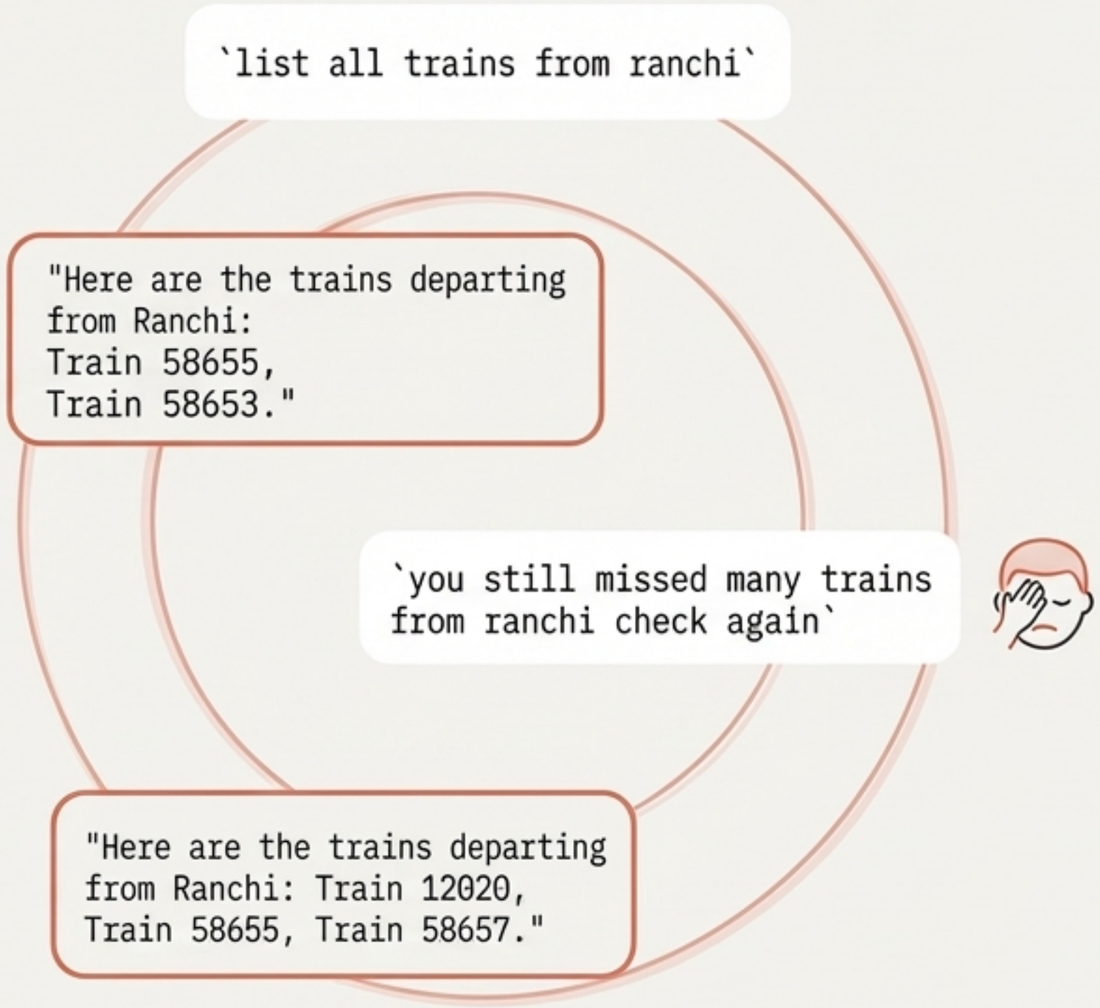
```
TRAIN NO: 57254
TRAIN NAME: Vijayawada Bhadrachalam Passenger
CODE | STATION NAME | ARR | DEP | DAY
-------------------------------------
BZA | VIJAYAWADA JN| --- | 08:00:00| 1
RYP | RAYANAPAD | 08:18:00| 08:19:00| 1
...
```

Search Chunk (For Indexing)

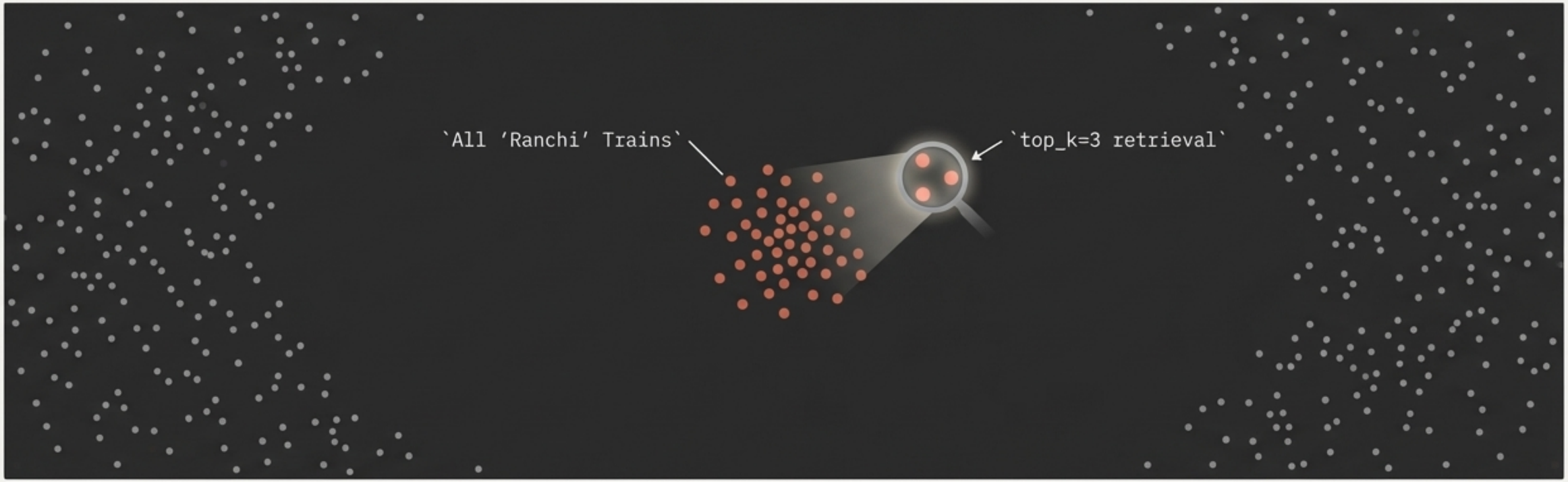
```
Train Number 57254 named Vijayawada
Bhadrachalam Passenger. Route stops at:
Vijayawada, Rayanapads, Kondapalli, Cheruvu
Madhavaram, Gangineni, Errupalem...


TRAIN NO: 57254
TRAIN NAME: Vijayawada Bhadrachalam Passenger
CODE | STATION NAME | ARR | DEP | DAY
...
```

The Final Trial: The Aggregation Goliath

The most difficult challenge emerged when we asked the system to aggregate data. A simple query to "list all trains from Ranchi" produced a frustratingly incomplete list. The user had to repeatedly ask, and the system could only ever return a few results at a time.



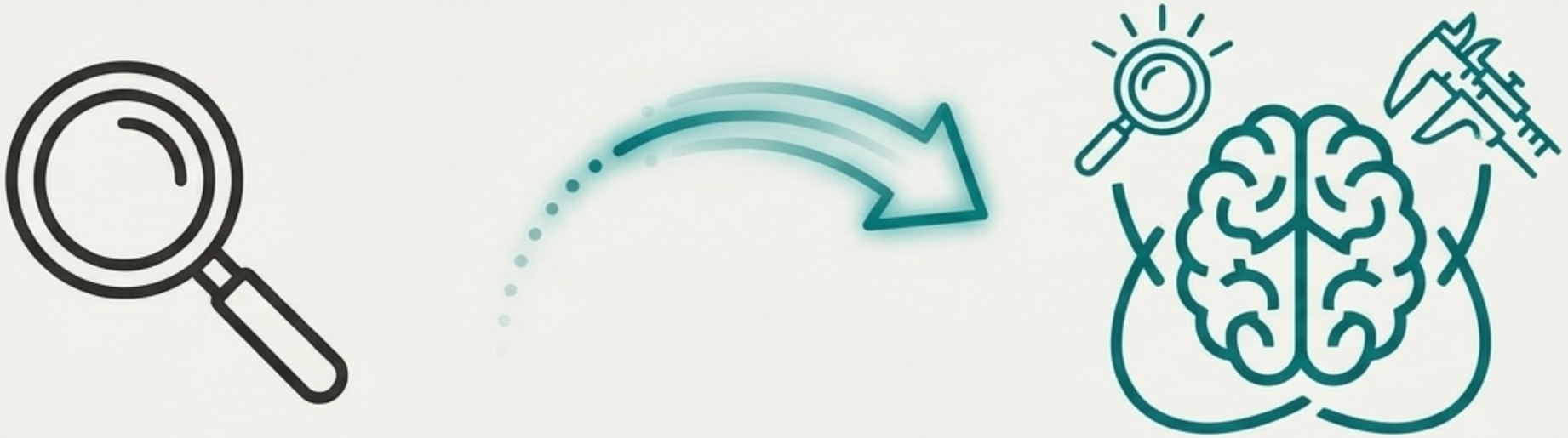
Our Weapons Are Not Enough



The Wall - We diagnosed the root cause: Vector search is built for `top-k` retrieval. It is fundamentally designed to find the *best* few matches, not *all* of them.

The Realization - No amount of prompt engineering, hybrid search, or enriched chunking can solve this problem. Pure Retrieval-Augmented Generation has reached its limit. **We don't need a better retriever; we need a different approach.**

The Paradigm Shift: From Retriever to Reasoner



Retriever

Reasoner

The solution wasn’t to build a better search tool. It was to build a smarter manager with a toolbox. Instead of a single system that retrieves text for every query, we need an agent that can reason about the user’s intent and delegate the task to the right specialist tool.

The Modern Solution: An Agent with a Toolbox

Tool A: The Vector Lens

`sentence-transformers`, `PyTorch`

Vague, semantic queries.

Example: 'Morning trains to Delhi.'

Gemini LLM Agent (The Manager)

Tool B: The Data Caliper

`pandas` DataFrame

Precise, structured queries.

Example: 'List all trains from Ranchi.'

The Agent in Action

How the new system handles the query `list all trains from ranchi`.

"List all trains from Ranchi."

Agent's Reasoning

"The user wants a 'list all' which is an aggregation task. The Vector Lens is bad at this. I must use the Data Caliper tool."

Agent's Action

Selects Pandas Tool.

Executed Code:

```
df[df['station_name'] == 'RANCHI']['train_name'].unique()
```

Result: 100% Complete

```
Ranchi-Howrah Shatabdi
Ranchi Lohardaga Passenger
Ranchi Chopan Express
Ranchi New Delhi Rajdhani
Ranchi-Patna Jan Shatabdi
...
(Complete and Accurate List)
```

The Evolutionary Path to Precision

Vanilla RAG

Hybrid RAG

Limitation: Fails on IDs and keywords.

Limitation: Fails on aggregation.

Capability: Solves all previous challenges.

Lessons from the Gauntlet

Use the Right Tool for the Job

Vectors are for semantic similarity, not database lookups. Use deterministic methods like dictionary lookups for IDs.

Guide, Don’t Just Trust

Raw data can confuse embedding models. Use 'Enriched Chunking' to inject descriptive headers and guide the AI's focus.

Recognize RAG’s Kryptonite

Retrieval-based systems are inherently bad at aggregation (counting, listing all). Don't fight the architecture; add a structured tool.

Think Agentically

The most robust AI systems don't rely on a single monolithic model. They are agents that reason about a user's intent and delegate tasks to a specialized toolbox.